



JECRCTM
UNIVERSITY
BUILD YOUR WORLD

Computer Programming in C++

Unit-V C++ Overview

Introduction



- Many C++ I/O features are object-oriented
 - Use references, function overloading and operator overloading
- C++ uses type safe I/O
 - Each I/O operation is automatically performed in a manner sensitive to the data type
- Extensibility
 - Users may specify I/O of user-defined types as well as standard types
- Stream:-
 - A transfer of information in the form of a sequence of bytes



Introduction



JECRC
UNIVERSITY
BUILD YOUR WORLD

➤ I/O Operations:-

- Input: A stream that flows from an input device (i.e.: keyboard, disk drive, network connection) to main memory
- Output: A stream that flows from main memory to an output device (i.e.: screen, printer, disk drive, network connection)

➤ Low-level I/O:-

- Unformatted
- Individual byte unit of interest
- High speed, high volume, but inconvenient for people

➤ High-level I/O:-

- Formatted
- Bytes grouped into meaningful units: integers, characters, etc.
- Good for all I/O except high-volume file processing



JECRC Foundation

Stream

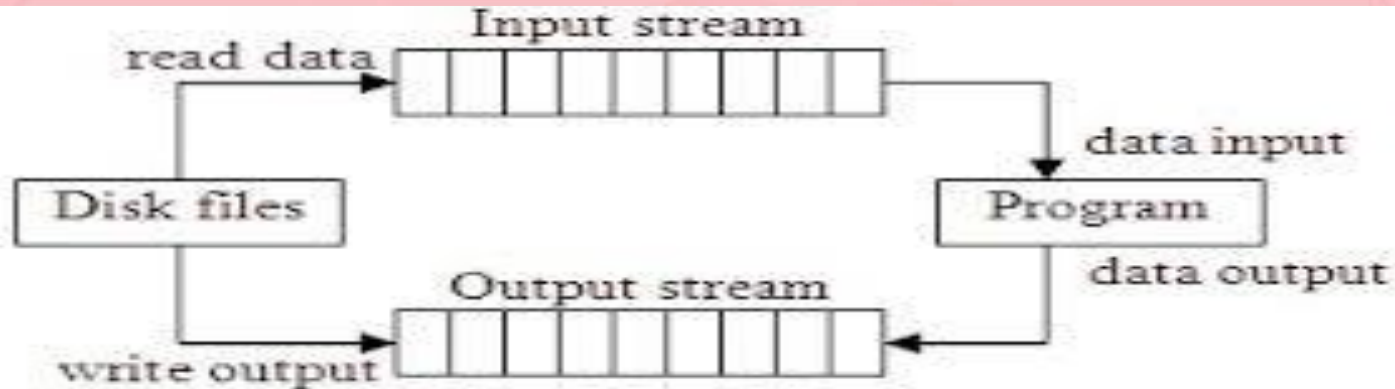
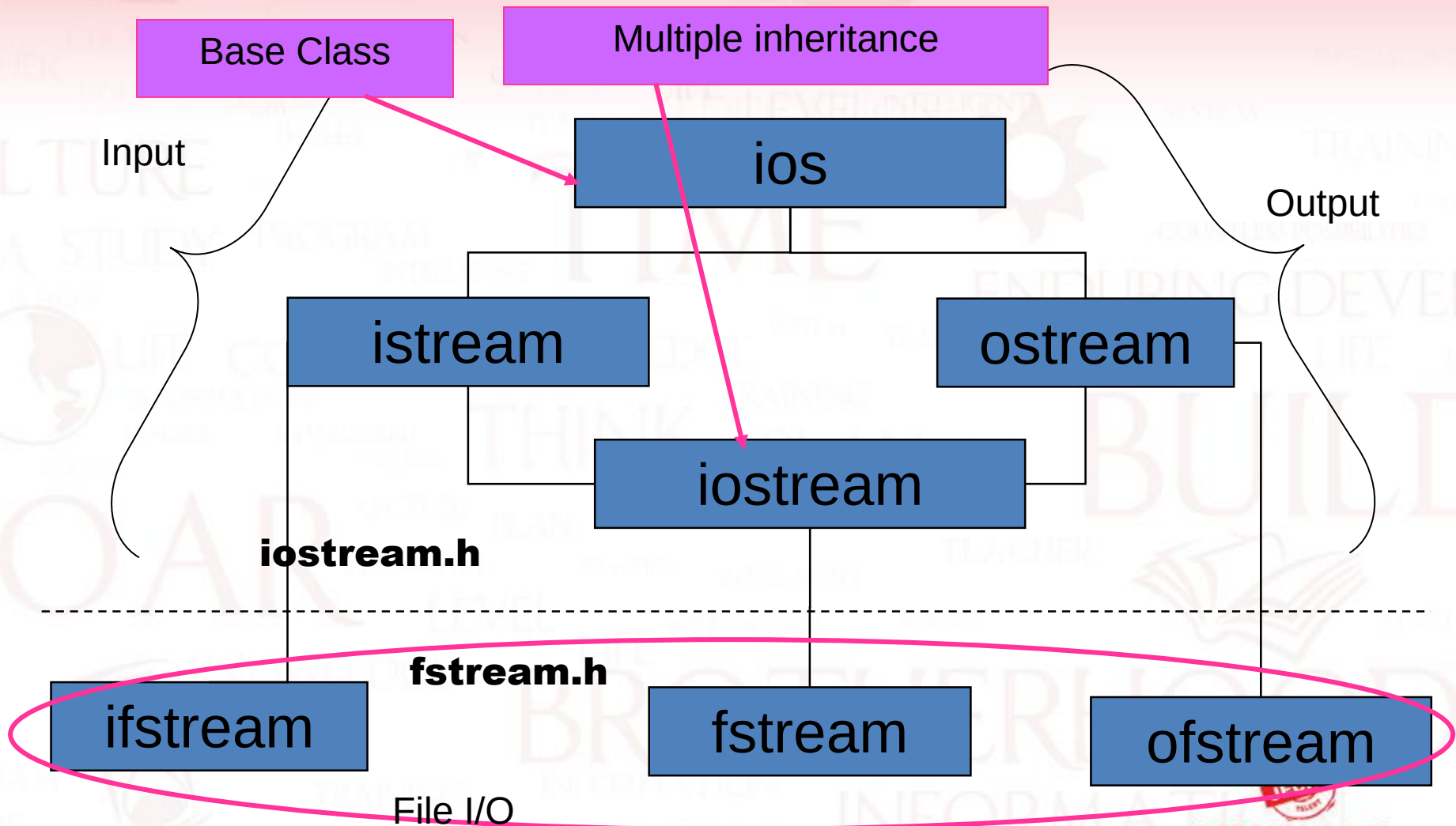


Fig : File input and output streams

Stream Classes:-The C++ I/O system contains a hierarchy of classes that are used to define various streams to deal with both the console and disk files. These classes are called stream. These classes are declared in the header file `iostream`. The file should be included in all programs that communicate with the console unit.



File Class Hierarchy



File Class Hierarchy



Class name	Contents
ios(General input/output stream class)	Contains basic facilities that are used by all other input and output classes Also contains a pointer to buffer object(streambuf object) Declares constants and functions that are necessary for handling formatted input and output operations
istream(input stream)	Inherits the properties of ios Declares input functions such as get(),getline() and read() Contains overloaded extraction operator>>
ostream(output stream)	Inherits the property of ios Declares output functions put() and write() Contains overloaded insertion operator <<
iostream (input/output stream)	Inherits the properties of ios stream and ostream through multiple inheritance and thus contains all the input and output Functions
streambuf	Provides an interface to physical devices through buffer Acts as a base for filebuf class used ios files

Table:- Stream classes for console operations



File Class Hierarchy



- **ios:**
 - **istream** and **ostream** inherit from **ios**
 - **iostream** inherits from **istream** and **ostream**.
- << (left-shift operator)
 - Overloaded as *stream insertion operator*
- >> (right-shift operator)
 - Overloaded as *stream extraction operator*
 - Both operators used with **cin**, **cout**, **cerr**, **clog**, and with user-defined stream objects
- **istream:** input streams
 - cin >> someVariable;**
cin knows what type of data is to be assigned to **someVariable** (based on the type of **someVariable**).
- **ostream:** output streams
 - cout << someVariable;**
cout knows the type of data to output
 - cerr << someString;**
Unbuffered - prints **someString** immediately.
 - clog << someString;**
Buffered - prints **someString** as soon as output buffer is full or flushed



File Class Hierarchy



➤ Stream O/P:-

- **ostream**: performs formatted and unformatted output
 - Uses **put** for characters and **write** for unformatted characters
 - Output of numbers in decimal, octal and hexadecimal
 - Varying precision for floating points
 - Formatted text outputs
 - **Stream Insertion Operator:-**
- << is overloaded to output built-in types
 - Can also be used to output user-defined types
 - **cout << '\n';**
 - Prints newline character
 - **cout << endl;**
 - **endl** is a stream manipulator that issues a newline character and flushes the output buffer
 - **cout << flush;**
 - **flush** flushes the output buffer



Cascading Stream-Insertion/ Extraction Operators



JECRC
UNIVERSITY
BUILD YOUR WORLD

- `<<` : Associates from left to right, and returns a reference to its left-operand object (i.e. **cout**).

- This enables cascading

cout `<<` "How" `<<` " are" `<<` " you?";

Make sure to use parenthesis:

cout `<<` "1 + 2 = " `<<` (1 + 2);

NOT

cout `<<` "1 + 2 = " `<<` 1 + 2;

Output of char * Variables

`<<` will output a variable of type **char *** as a string

To output the address of the first character of that string, cast the variable as type **void ***



JECRC Foundation

File Class Hierarchy



➤ Input Steam

>> (stream-extraction)

- Used to perform stream input
- Normally ignores whitespaces (spaces, tabs, newlines)
- Returns zero (**false**) when **EOF** is encountered, otherwise returns reference to the object from which it was invoked (i.e. **cin**)

- This enables cascaded input
- **cin >> x >> y;**

>> controls the state bits of the stream

- **failbit** set if wrong type of data input
- **badbit** set if the operation fails



Member Functions (get and getline)

- **cin.get():** inputs a character from stream (even white spaces) and returns it
- **cin.get(c):** inputs a character from stream and stores it in **c**
- **ch = cin.get();**
// inputs & returns one character
 - even if it is whitespace -- returns the whitespace character
 - returns the EOF when end of file reached
- **cin.eof()** returns true (1) when user enters ctrl-z
- **cin.get(ch);** // new version
 - reads value from stream into **ch**
 - returns 0 when eof encountered
 - otherwise returns reference to stream object (for cascading)
- **cin.get (name, 30, '\n');** // 3rd version
 - 30 specifies max number of characters
 - '\n' terminates input
 - '\n' not consumed

Unformatted I/O & getline Function

- `cin.getline(name,30,'\n');`
 - Similar to `cin.get( ... )`
 - Difference is that the `getline` version will "consume" the `'\n'` (or whatever delimiter is specified)
 - Need to be aware of using `cin.ignore` if we know `'\n'` will still be in the stream
 - **Unformatted I/O:-**
 - Performed with **read** and **write** member functions
 - They take two parameters
 - an array of char or a buffer
 - a number which tells how many characters
- `cin.read(buffer,20);`**
`cin.write (buffer,15);`

Formatted Console I/O Operations



- C++ supports a number of features that could be used for formatting the output. These features include:
 - ios class functions and flags.
 - Manipulators.
 - User-defined output functions.
- The ios class contains a large number of member functions that would help us to format the output in a number of ways. The most important ones among them are listed in Table.



Formatted Console I/o Operations



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

Function	Task
Width()	To specify the required field size for displaying an output value.
precision()	To specify the number of digits to be displayed after the decimal point of a float value.
fill()	To specify a character that is used to fill the unused portion of a field.
setf()	To specify format flags that can control the form of output display(such as left-justification and right-justification)
unsetf()	To clear the flags specified.

Manipulators



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

Manipulators	Equivalent ios function
<code>setw()</code>	<code>width()</code>
<code>setprecision()</code>	<code>precision()</code>
<code>setfill()</code>	<code>fill()</code>
<code>setiosflags()</code>	<code>setf()</code>
<code>resetiosflags()</code>	<code>unsetf()</code>

Manipulators



- Some examples of manipulators are given below:

```
cout<<setw(10)<<12345;
```

- This statement prints the value 12345 right-justified in a field width of 10 characters.

- The output can be made left-justified by modifying the statement as follows:

```
cout<<setw(10)<<setiosflags(ios::left)<<12345;
```



Manipulators



- One statement can be used to format output for two or more values.

For example, the statement

```
cout<<setw(5)<<setprecision(2)<<1.2345  
<<setw(10)<<setprecision(4)<<sqrt(2)  
<<setw(15)<<setiosflags(ios::scientific)  
<<sqrt(3)<<endl;
```

- Will print all the three values in one line with the field size of 5, 10, and 15 respectively.



Manipulators



➤ There is a major difference in the way the manipulators are implemented as compared to the ios member functions. The ios member function return the previous format state which can be used later. In case, we need to save the old format states, we must use the ios member function rather than the manipulators.



Manipulators



JECRC
UNIVERSITY
BUILD YOUR WORLD

Example:

```
cout.precision(2);//previous state
```

```
int p=cout.precision(4);//current state;
```

When these statements are executed, p will hold the value of 2(previous state) and the new format state will be 4. We can restore the previous format state as follows:

```
cout.precision(p)//p=2
```



JECRC Foundation

Detail of File Stream Classes



Class	Contents
filebuf	Its purpose is to set the file buffers to read and write. Contains Openprot constant used in the open() of file stream classes. Also contain close() and open() as members.
fstreambase	Provides operations common to file streams. Serves as a base for fstream , ifstream and ofstream class. Contains open() and close() functions.
ifstream	Provides input operations. Contains open() with default input mode. Inherits the functions get() , getline() , read() , seekg() , tellg() functions from istream .
ofstream	Provides output operations. Contains open() with default output mode. Inherits put() , seekp() , tellp() and write() functions from ostream .
fstream	Provides support for simultaneous input and output operations. Contains open with default input mode. Inherits all the functions from istream and ostream classes through iostream .

File Handling



Opening Files using Constructor:

While using constructor for opening files, filename is used to initialize the file stream object. This involves the following steps

- (i) Create a file stream object to manage the stream using the appropriate class i.e the class **ofstream** is used to create the output stream and the class create the input stream.

ifstream to

- (ii) Initialize the file object using desired file name.

For example, the following statement opens a file named “results” for output:

```
ofstream outfile(“results”); //output only
```

This create outfile as an **ofstream** object that manages the output stream.

Similarly ,the following statement declares infile as an **ifstream** object and attaches it to the file data for reading (input).

```
ifstream infile (“data”); //input only
```

The same file name can be used for both reading and writing data



File Handling



Working with single file:-

constructor function

```
#include <iostream.h>
#include <fstream.h>
int main()
{
    ofstream outf("ITEM");
    cout << "enter item name: ";
    char name[30];
    cin >> name;
    outf << name << "\n";
    cout << "enter item cost : ";
    float cost;
```

```
    cin >> cost;
    outf << cost << "\n";
    outf.close();
    ifstream inf("item");
    inf >> name;
    inf >> cost;
    cout << "\n";
    cout << "item n//Creating
files with ame : " <<
name << "\n";
    cout << "item cost: " <<
cost << "\n";
    inf.close();
    return 0;
}
```



File Handling



Opening Files using open()

The function **open()** can be used to open multiple files that uses the same stream object. For example to process a set of files sequentially, in such case a single stream object can be created and can be used to open each file in turn. This can be done as follows;

File-stream-class stream-object;

stream-object.**open** ("filename");

The following example shows how to work simultaneously with multiple files



File Handling



Working with multiple file:-

//Creating files with open() function

```
#include <iostream.h>
```

```
#include<fstream.h>
```

```
int main()
```

```
{
```

```
    ofstream fout;
```

```
    fout.open("country");
```

```
    fout<<"United states of America  
    \n";
```

```
    fout<<"United Kingdom";
```

```
    fout<<"South korea";
```

```
    fout.close();
```

```
    fout.open("capital");
```

```
    fout<<"Washington\n";
```

```
    fout<<"London\n";
```

```
    fout<<"Seoul \n";
```

```
    fout.close();
```

```
const int N =80;
```

```
char line[N];
```

```
ifstream fin;
```

```
fin.open("country");
```

```
cout<<"contents of country file \n";
```

```
while (fin)
```

```
{
```

```
    fin.getline(line,N);
```

```
    cout<<line; }
```

```
    fin.close();
```

```
    fin.open("capital");
```

```
    cout<<"contents of capital file";
```

```
    while(fin)
```

```
{
```

```
    fin.getline(line,N);
```

```
    cout<<line; }
```

```
    fin.close();
```

```
    return 0; }
```



File pointers



- Each file has two associated pointers known as the file pointers.

One of them is called the input pointer or get pointer.

The get pointer specifies a location from which the current reading operation is initiated

Other is called the output pointer or put pointer.

The put pointer specifies a location from where the current writing operation is initiated

We can use these pointers to move through the files while reading or writing.

The input pointer is used for reading the contents of a given file location and the output pointer is used for writing to a given file location.

Functions for manipulation of file pointers

`seekg()` Moves get pointer (input) to a specified location.

`seekp()` Moves put pointer (output) to a specified location.

`tellg()` Gives the current position of the get pointer.

`tellp()` Gives the current position of the put pointer.





JECRCTM
UNIVERSITY
BUILD YOUR WORLD



JECRCTM
UNIVERSITY
BUILD YOUR WORLD

Thank You!



JECRC Foundation